

Demystifying NVSHMEM

Abstract—NVSHMEM is NVIDIA’s OpenSHMEM-based PGAS communication library for GPU clusters. It is increasingly used as a substrate for GPU-initiated communication, yet much of its practical behavior is spread across documentation, source code, and application-specific folklore rather than a single systems-oriented explanation. This paper distills a prior in-depth thesis into a conference-length study of NVSHMEM’s programming model, implementation structure, and performance characteristics. We first explain the design choices that distinguish NVSHMEM from CPU-centric MPI and collective-oriented NCCL, including symmetric memory, device-side one-sided communication, and explicit memory ordering. We then analyze the initialization path, symmetric heap management, transport selection, point-to-point fast and slow paths, and collective implementations. Finally, we consolidate benchmark observations and two case studies. Our main findings are that NVSHMEM is strongest when applications need GPU-initiated, irregular, or fused communication; that its built-in collectives are often constrained by conservative kernel structure and can lag NCCL; and that modern applications such as DeepEP use NVSHMEM primarily as a low-level substrate rather than as a drop-in collective library. These results clarify when NVSHMEM is the right abstraction, where its current bottlenecks lie, and which design directions matter most for future GPU-centric communication runtimes.

Index Terms—NVSHMEM, PGAS, GPU communication, one-sided communication, collective communication

I. INTRODUCTION

The rise of distributed deep learning and GPU-accelerated scientific computing has made GPU communication a central systems challenge. Distributed deep learning increasingly relies on different types of parallelism, such as data, tensor, and pipeline parallelism, to scale training and inference across many GPUs and often multiple nodes [1]. HPC applications likewise rely on distributed decompositions to scale simulations beyond a single device [2]–[4]. In both settings, performance depends not only on compute throughput, but also on how efficiently data can be exchanged over interconnects such as PCIe, NVLink, and InfiniBand (IB). Communication libraries must therefore provide high bandwidth, low latency, and abstractions that integrate naturally with GPU execution. The NVIDIA Collective Communication Library (NCCL) [5] has emerged as a widely used solution in this space, providing optimized collective operations such as AllReduce, AllGather, and ReduceScatter, and serving as the recommended backend for most CUDA-based distributed machine learning frameworks such as PyTorch [6], Megatron-LM [7], and vLLM [8].

However, NCCL’s original programming model is primarily host-driven: the CPU enqueues collective operations, and the library selects algorithms and schedules. While effective for bulk-synchronous collectives, this model is less suitable for GPU-initiated communication. Such patterns arise when com-

munication is data dependent, restricted to subsets of threads, or when avoiding CPU round trips is critical. Examples include stencil halo exchange, irregular graph communication, sparse and expert-parallel workloads, and custom kernels that tightly couple computation and data movement. While NCCL now supports device-initiated communication through its new device API [9], reducing a key historical difference, its abstractions remain centered around communicators and collective-oriented primitives.

NVSHMEM [10], [11] addresses this gap through a complementary programming model. It exposes a partitioned global address space (PGAS) directly to GPU code, enabling CUDA kernels to access symmetric memory on remote processing elements via one-sided put/get operations, atomics, and explicit synchronization primitives. Rather than replacing NCCL, NVSHMEM enables communication patterns that are difficult to express with host-launched collectives, including irregular exchanges and fine-grained overlap.

Despite NVSHMEM’s growing importance, many of its key behaviors are not apparent from the API alone. How does NVSHMEM realize symmetric memory on modern GPUs? Which communication paths truly avoid CPU involvement, and under what conditions do they apply? How do teams and collective launch requirements shape the implementation and performance of the built-in collective layer? In this paper, we answer these questions through a detailed source code analysis that demystifies NVSHMEM from the programming model down to its runtime internals and transport mechanisms, and relates those design choices to observed performance.

Our analysis primarily targets NVSHMEM version 3.3.9. Although future releases may change specific features or implementation details, the core architectural components, such as symmetric memory and communication paths are fundamental to NVSHMEM’s design and are likely to remain largely unchanged. We therefore expect the main insights of this paper to remain relevant for future versions as well.

This paper makes the following contributions:

- An in-depth implementation study of the NVSHMEM source code that demystifies its core components, including symmetric memory management, initialization, communication paths, synchronization semantics, and the design of the collectives.
- A comprehensive performance characterization based on the NVSHMEM test suite, covering latency and bandwidth across both intra-node and inter-node settings.
- A comparison against NCCL through a Jacobi solver case study, illustrating the potential impact of these two different programming models.

- An analysis of application integration through DeepEP, a state-of-the-art communication library for Mixture-of-Experts (MoE) models built on top of NVSHMEM.

II. BACKGROUND

To better understand NVSHMEM, it is first necessary to introduce several core concepts that shape its design. NVSHMEM builds on three main ideas: the limitations of host-initiated communication, the motivation for device-initiated communication, and the PGAS/OpenSHMEM programming model based on symmetric memory and one-sided operations.

NVIDIA Collective Communication Library (NCCL) has become the standard communication library for multi-GPU systems, but its conventional interface remains largely **host-initiated** [5]. In particular, communicator setup and the launch of collective operations are performed from the CPU, which then schedules NCCL kernels onto CUDA streams. While this model is highly optimized for regular collective communication, it still places the host in the communication control path and can introduce coordination overhead between computation and communication.

This limitation motivates **device-initiated communication**, where communication and synchronization are launched directly from GPU code rather than through a host-managed control path. This reduces CPU involvement and enables finer-grained overlap between communication and computation. These benefits are especially important for latency-sensitive and strongly scaled GPU workloads, and motivate interfaces such as NVSHMEM that support GPU-side communication primitives and GPU-initiated network operations that bypass CPU proxy threads completely [10], [12], [13].

Peer-to-peer (P2P) memory access refers to the ability of one GPU to directly access another GPU’s memory over an on-node interconnect, such as PCIe or NVLink. This is important for NVSHMEM as it enables low-overhead remote memory access within a node. **GPUDirect RDMA (GDRDMA)** [14] and **GPUDirect Async** [12] make this model practical across nodes. GDRDMA allows an RDMA-capable NIC to access GPU memory directly, avoiding extra copies through host memory. **GPUDirect Async Kernel-Initiated (GDA-KI)** [9], [15] extends this by allowing GPU kernels to control network operations without CPU intervention. **InfiniBand GPUDirect Async (IBGDA)** [12] is the InfiniBand-specific realization of GDA-KI used by NVSHMEM, allowing GPU kernels to prepare and post NIC work directly.

The **Partitioned Global Address Space (PGAS)** breaks the coupling inherent in NCCL and MPI [16] by exposing a logically shared address space partitioned across the processing elements (PEs). In NVSHMEM, a processing element (PE) is an OS process that is mapped to one GPU; since version 2.4.1, NVSHMEM also provides limited support for multiple PEs per GPU. This model underlies OpenSHMEM [17] and is adapted by NVSHMEM for GPU-based systems. In NVSHMEM, communication is built on **symmetric memory**, consisting of a **symmetric heap** on each PE and **symmetric objects** allocated with the same type, size, and layout across all

PEs. This structure allows remote PEs to access those objects through one-sided operations such as `put` and `get`, where the initiator specifies the source, destination, and location without explicit receives from the target PE. This enables asynchronous progress and, in many cases, direct placement into the destination’s symmetric memory buffer.

III. RELATED WORK

To understand NVSHMEM’s role in the broader high-performance computing (HPC) landscape, we compare it with existing GPU communication frameworks and libraries.

CUDA-aware MPI [18] extends the standard MPI programming model [19] by allowing MPI routines to operate directly on GPU-resident buffers, avoiding explicit application-level staging through host memory. This makes it an effective path for porting existing distributed-memory HPC applications to GPU clusters while preserving familiar rank-based, two-sided communication semantics [19]. However, CUDA-aware MPI remains fundamentally host initiated.

NCCL, discussed previously, is the main alternative to NVSHMEM for GPU communication. It is optimized for dense and regular collective communication, using multi-channel execution, pipelining, and protocol specialization to achieve high bandwidth and low latency. This makes it the natural baseline for distributed deep learning and other bulk-synchronous workloads. However, its conventional interface remains largely host-centric. Recent NCCL Device API work reduces this gap by introducing GPU-initiated communication, but the main distinction remains unchanged: NCCL is fundamentally collective-centric, whereas NVSHMEM is designed around fine-grained one-sided communication and still shows an advantage for such operations in the evaluation of NCCL’s GPU-Initiated Networking (GIN) interface [9].

UPC++ [20] and **GASNet-EX** [21] provide portable PGAS-style communication and share with NVSHMEM the idea of one-sided access to a partitioned address space. Their portability and C++ integration make them attractive for many host-driven HPC applications. The main distinction, however, lies in where communication is initiated. While UPC++ and GASNet-EX can support GPU memory through device-aware transports, the control path is still typically managed by host code. NVSHMEM, by contrast, exposes device-callable operations that allow CUDA threads, warps, or thread blocks to initiate communication directly from the GPU.

rocSHMEM [22] and **Intel SHMEM** [23] bring OpenSHMEM-style, GPU-oriented one-sided communication to AMD ROCm and Intel oneAPI/SYCL systems, respectively. Together, they show that device-initiated PGAS communication is becoming a broader vendor trend rather than a feature unique to NVIDIA. However, each implementation is necessarily shaped by its own compiler stack, memory model, interconnects, and transport mechanisms.

IV. NVSHMEM OVERVIEW

Before analyzing NVSHMEM’s internals, we first summarize the main concepts that shape how it is used in practice.

TABLE I
BREAKDOWN OF NVSHMEM API GROUPS BY AVAILABILITY AND FUNCTIONALITY IN THE SOURCE CODE.

API group	Host/Device Availability	Description
Setup / exit	Mixed	Initialize and finalize the library, bootstrap jobs, and handle exceptional termination.
Memory management	Host-side	Allocate, free, and align symmetric objects in the symmetric heap, provide registration functionality for selected local buffers.
Team management	Mixed, mostly host-side	Define communication subgroups, query team-relative PE identities, translate PE numbers across teams, and create or destroy user-defined teams.
One-sided RMA	Both	Provide one-sided data movement through <code>put</code> , <code>get</code> , scalar <code>p/g</code> , etc.
Atomics	Both	Provide remote atomic read-modify-write operations such as <code>fetch</code> , <code>add</code> , <code>compare-and-swap</code> , and bitwise updates on symmetric data.
Memory ordering	Both	Provide ordering and completion primitives such as <code>fence</code> and <code>quiet</code> .
Synchronization	Mixed, mostly device-side	Provide value-based synchronization through <code>wait</code> and <code>test</code> operations.
Collectives	Both	Provide coordinated multi-PE operations such as <code>allreduce</code> , and <code>broadcast</code> .

A. Teams

Teams are NVSHMEM’s mechanism for defining subsets of processing elements (PEs) that participate in communication. Teams play a role analogous to communicators in MPI and NCCL. The default team is `NVSHMEM_TEAM_WORLD`, which contains all PEs in the program and is implicitly used whenever a team argument is omitted.

An important consequence is that teams carry semantic and runtime constraints. Collective operations on a team are collective over exactly the PEs in that team, and operations that use team-relative indexing must translate PE numbers correctly between parent and child teams. Moreover, a team cannot safely be used by multiple concurrent collective invocations from the same PE, because the runtime associates a single set of internal resources, such as synchronization state and scratch storage, with each team.

B. API and Dual-Interface Design

This dual-interface design is one of NVSHMEM’s key strengths, but it also makes the library substantially more complex than communication libraries built around a single control path. This complexity is compounded by the fact that the split is not uniform across the API: some components remain fundamentally host-managed, whereas the core communication path is exposed on both the CPU and the GPU. To clarify this structure, we group the APIs into functional subsets and summarize their purpose and availability in Table I.

In addition, the naming convention distinguishes the core interface from NVSHMEM-specific extensions: `nvshmem_` generally denotes the standard API, whereas `nvshmemx_` is used for extensions such as stream-ordered operations and threadgroup-scoped variants

Within the device-side interface, NVSHMEM also distinguishes different **execution scopes**. The default device APIs are thread-scoped, meaning that a single CUDA thread issues the operation, while many `nvshmemx_` extensions additionally provide `_warp` and `_block` forms in which a whole warp or thread block cooperatively performs one logical communication operation.

Besides Table I, the API groups are described below. We present a high-level overview of the functionality of each group. For a complete description, we refer the reader to the official NVSHMEM documentation [10].

1) *Setup / exit*: This group brings NVSHMEM up and tears it down. It covers initialization, bootstrap through MPI or unique IDs, and finalization, and is therefore primarily responsible for creating the runtime state required by all later operations. In the source, this group is mostly host-side, with `nvshmem_global_exit` as the main exception that is also exposed on the device.

2) *Memory management*: The memory-management APIs allocate and manage symmetric objects in the symmetric heap. They include routines such as `nvshmem_malloc`, `nvshmem_align`, and `nvshmem_free`, together with buffer-registration functionality for selected local buffers. These routines remain host-driven because they manipulate global runtime state and collective memory layout.

3) *Team management*: Team-management APIs define communication subgroups and provide the rank-translation needed to operate within them. Query functions such as `nvshmem_team_my_pe` and `nvshmem_team_n_pes` are available on both host and device, while team creation and destruction remain host-managed.

4) *One-sided remote memory access (RMA)*: The RMA group is the core one-sided communication interface of NVSHMEM. It includes typed and untyped `put/get` operations, scalar `p/g`, strided `iput/iget`, and nonblocking variants. The API appears in several generalized naming forms: typed routines follow patterns such as `nvshmem_<TYPENAME>_{put,get}`, size-specialized routines follow forms such as `nvshmem_{put,get}<SIZE>`, and untyped byte-oriented routines use names such as `nvshmem_{putmem,getmem}`. Scalar `p/g` operations cover single-element access, while `iput/iget` provide shorthand for strided communication patterns. This group is fully dual-interface and is also extended with device-side warp- and block-scoped variants and host-side on-stream forms.

5) *Atomics*: Atomic APIs provide remote read-modify-write operations on symmetric data, including operations such as fetch and add. Their purpose is to support fine-grained remote coordination on individual memory locations. The API follows generalized naming forms such as `nvshmem_<TYPENAME>_atomic_<op>` for non-fetching operations and `nvshmem_<TYPENAME>_atomic_fetch_<op>` for fetching variants, together with special forms such as `compare_swap`, `swap`, and `fetch`. This group is exposed on both the host and the device API.

6) *Memory ordering*: The memory-ordering interface is centered around a small number of key routines. `nvshmem_fence` enforces ordering of previously issued communication operations to the same destination PE, but does not by itself guarantee their completion. `nvshmem_quiet` is stronger: it waits until previously issued NVSHMEM communication operations from the calling context have completed and become visible at the destination. On the host side, NVSHMEM also provides `nvshmemx_quiet_on_stream`, which places this completion semantics into CUDA stream order.

7) *Synchronization*: The synchronization group lets a PE wait for or test remote progress based on the value of a symmetric object or signal variable. This includes the `wait` and `test` families as well as signal-based waiting operations. In contrast to RMA and collectives, this group is less uniformly dual-interface: the ordinary `wait/test` routines are primarily device-side, while the host mainly accesses this functionality through on-stream variants.

8) *Collectives*: The collective API group provides team-wide communication and synchronization operations, including `barrier`, `sync`, `broadcast`, `all-to-all`, `fcollect`, `allreduce`, and `reducescatter`. These APIs are available on both host and device, and the device-side interface further supports `thread`-, `warp`-, and `block`-scoped variants. In the PGAS model, `fcollect` is the counterpart of MPI `allgather`. In addition, NVSHMEM memory-management and team-creation functions are also collective and internally invoke `nvshmem_barrier_all`.

V. MEMORY MANAGEMENT

Having outlined the overall design of NVSHMEM, we now examine its internals in more detail. We begin with how symmetric memory is realized and managed, as it is one of the core foundations enabling device-initiated communication.

NVSHMEM implements the symmetric heap on top of CUDA’s low-level virtual memory management (VMM) API, and alternative backends using system memory or pinned device memory can be selected via environment options. With VMM enabled, NVSHMEM implements memory management scheme that cleanly separates **virtual address (VA) reservation** from **physical memory allocation**. The symmetric heap’s VA range is reserved eagerly during initialization, while physical pages are committed on demand as the application allocates from the heap [24]. Figure 1 illustrates the key concepts discussed in this section.

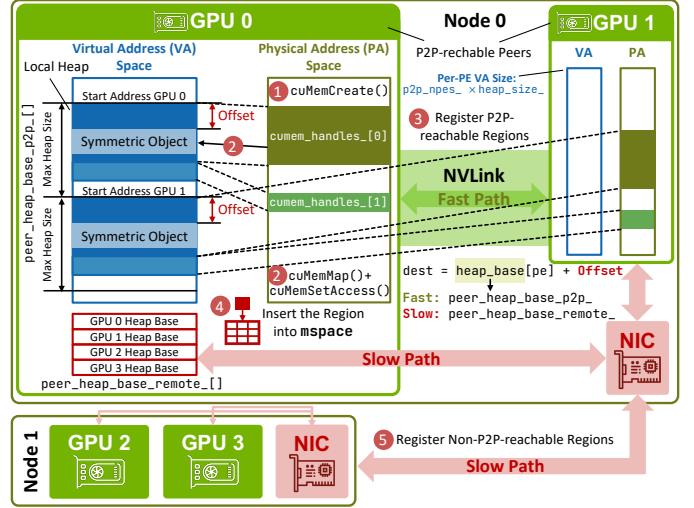


Fig. 1. Virtual-memory-based symmetric-heap setup in NVSHMEM for two nodes, each with two GPUs and one NIC. Remote symmetric addresses are computed by preserving an object’s offset within the local heap and applying it to the selected peer heap base, using either the P2P-mapped fast path or the transport-backed slow path. The numbered markers 1–5 show the main steps taken when new physical memory is allocated.

A. Initialization

During `nvshmem_init`, NVSHMEM establishes a symmetric heap abstraction for each PE. When CUDA VMM is enabled, it first determines the allocation granularity as the maximum of the value returned by `cuMemGetAllocationGranularity`, the runtime option `CUMEM_GRANULARITY`, and `NVSHMEMI_MAX_HANDLE_LENGTH`. It then selects a per-PE heap size as the larger of `MAX_MEMORY_PER_GPU` and the internal runtime overhead, rounded up to that granularity. The internal overhead includes bookkeeping structures and coalescing buffers used by operations such as `put` and `get`, while `MAX_MEMORY_PER_GPU`, which defaults to 128 GiB, sets an upper bound on the GPU memory that NVSHMEM may manage for symmetric allocations.

Each PE then reserves a contiguous VA range of size `p2p_npes_ × heap_size_` using `cuMemAddressReserve()`, where `p2p_npes_` is the number of P2P-reachable GPUs. The first segment is used for the local symmetric heap, and the remaining equal-sized segments are reserved for peer mappings. This gives each reachable peer a fixed VA segment in the local GPU address space. NVSHMEM then creates a host-side `mspace` allocator over the local segment without yet committing physical memory. After the transport layer is initialized, all PEs exchange their local heap base addresses so that remote memory access can later be established across nodes.

The reserved VA range covers only P2P-mappable peers. For non-P2P peers, NVSHMEM instead allgathers each PE’s `heap_base_` into a separate table, `peer_heap_base_remote_`, which stores the actual base address of every remote heap. The slow path uses this table

together with transport metadata and remote memory handles.

B. Memory Allocation & Registration

On top of the reserved VA range, NVSHMEM manages the symmetric heap with a simple host-side allocator implemented in the `mpace` class. It maintains three `std::map<void*, size_t>` structures: two for free chunks indexed by start and end address, and one for allocated chunks indexed by start address. This design makes coalescing straightforward since freeing a block allows the allocator to check both neighboring directions and merge adjacent free chunks when possible. Allocation is performed by `mpace::allocate` using a first-fit policy: the free list is scanned in address order, the first sufficiently large block is chosen, and it is either split or consumed entirely. All sizes are aligned to `NVSHMEMI_MALLOC_ALIGNMENT`, and the resulting symmetric address is returned to the user.

The allocator is deliberately simple and deterministic. It linearly scans the free list, and allocation is $O(n)$ in the number of free chunks, but this overhead is acceptable as symmetric allocations are typically executed in the initialization phase. In contrast to host memory allocators such as `jemalloc`, it avoids per-thread arenas, size classes, or other complex optimizations. The allocator is not internally thread-safe and instead relies on NVSHMEM’s thread model, especially `NVSHMEM_THREAD_SERIALIZED`, to avoid concurrent calls. Although it supports internal reallocation with device-to-device `cudaMemcpy()` when needed, NVSHMEM does not provide a public `realloc` API, reflecting the expectation that symmetric memory layouts remain mostly fixed after initialization.

NVSHMEM allocates physical GPU memory for the symmetric heap only on demand, when an allocation request cannot be satisfied from already-backed VA space. When a symmetric allocation request triggers growth of the heap, `allocate_physical_memory_to_heap()` performs the following steps:

- ① Create a physical memory handle with `cuMemCreate()`.
- ② Map the corresponding heap subrange with `cuMemMap()` and install access permissions with `cuMemSetAccess()` for the local and eligible peer GPUs.
- ③ Register the region with P2P transport.
- ④ Insert the region into the `mpace` allocator.
- ⑤ Register the region with network transports for communication with non-P2P-reachable peers.

NVSHMEM then records the allocation metadata (i.e., offset, size, etc.), exchanges the required handles across PEs, and synchronizes with a barrier.

C. Mapping and Remote Address Computation

NVSHMEM reserves a virtual address range large enough to cover the symmetric heaps of all P2P-reachable GPUs and places each peer heap at a fixed offset within that range. This organization is central to the design, because it allows remote addresses to be derived from heap-relative offsets rather than tracked through per-object metadata.

When new physical memory is added to the heap, NVSHMEM maps it so that peers can access it consistently. At a high level, it exports the local CUDA memory handle, exchanges these handles across PEs, and maps peer memory into the corresponding reserved virtual address segments. For non-P2P transports, the same region is additionally registered with the appropriate network interface.

Remote address computation follows the same symmetric-offset rule in both the fast and slow paths; the only difference is which per-PE base address is used and whether the resulting address can be dereferenced directly. In the P2P fast path, if `heap_base_` is the base of the local symmetric heap and `peer_heap_base_p2p_[remote_pe]` is the base at which the remote PE’s heap is mapped in the local VA space, then for a local symmetric pointer `dest_local` the corresponding remote address is

$$\text{dest_remote} = \text{peer_heap_base_p2p_}[remote_pe] + (\text{dest_local} - \text{heap_base_}) \quad (1)$$

NVSHMEM, therefore, preserves the object’s offset within the local heap and applies it to the remote heap base. For non-P2P-accessible GPUs, the same rule is applied using `peer_heap_base_remote_[remote_pe]`, and the computed address is passed to the transport layer together with the required memory handle, registration metadata, or NIC key. The same addressing scheme thus applies in both paths, but only the P2P case enables **direct pointers**, allowing remote symmetric data to be accessed through ordinary GPU loads and stores to mapped virtual addresses. Whenever the heap grows, NVSHMEM also refreshes device-resident metadata so that GPU kernels observe the updated layout correctly.

VI. ONE-SIDED COMMUNICATION

Having discussed how NVSHMEM manages memory, we now turn to another fundamental aspect of the library: one-sided communication. As noted in the previous section, NVSHMEM uses either a fast path or a slow path depending on whether the target peer is P2P-reachable. The following sections explain how data is moved between PEs in each case.

A. Fast Path: Direct GPU Memory Access

Fast-path eligibility. This decision is made by the host-side P2P transport logic, primarily in `nvshmemt_p2p_can_reach_peer`. For brevity, we focus on the most common case and omit less typical fallback paths, such as NVIDIA Management Library (NVML)-based handling. In this path, NVSHMEM verifies that the two PEs are on the same host, identifies the peer as a locally visible CUDA device, and uses `cudaDeviceCanAccessPeer` to determine whether direct peer access is supported. If P2P access is available, it additionally queries `cudaDevP2PAttrNativeAtomicSupported` to determine whether native GPU atomics can be enabled. Only then does NVSHMEM populate `peer_heap_base_p2p_[pe]` for that PE.

Device-side RMA Once the peer heap is directly mapped, device-side RMA reduces to ordinary memory access on

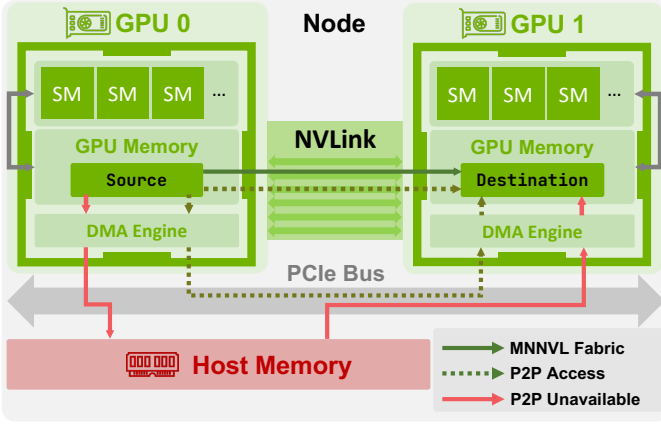


Fig. 2. Device-side RMA fast path. The figure shows the routes under which the mapped fast path can be enabled: **MNNVL fabric**, where data is transferred directly over multi-node NVLink, and **P2P access**, where CUDA peer access exposes the standard NVLink/PCIe peer-memory path. When neither route is available, NVSHMEM falls back to the slow path, where communication is handled indirectly through the host.

the computed remote address from Equation 1. For scalar operations, helpers such as `nvshmemi_p` and `nvshmemi_g` first check whether the target PE has a mapped heap base and then issue a direct store or load on the resulting remote pointer. Similarly, bulk interfaces such as `nvshmemi_put_threadgroup` perform threadgroup-wide `memcpy` operations to or from that mapped address.

Host-side RMA Host-side RMA uses the same mapped address space, but the execution model is different since the host orchestrates CUDA copy operations. Public APIs such as `nvshmem_put` and `nvshmemx_put_on_stream` delegate to `nvshmemi_prepare_and_post_rma`, which selects a mapped P2P path when the peer heap is locally accessible. In that case, NVSHMEM uses CUDA copy operations such as `cudaMemcpyAsync` between local and remote device pointers. For device-local symmetric buffers, it may use specialized helpers such as `nvshmemi_p2p_rma_optimized`, which select explicit copy kinds for stream-ordered and single-word operations.

B. Slow Path: IBGDA and Proxy Execution

The slow path is used when the target PE’s symmetric heap is not directly P2P-mapped into the local GPU address space, i.e., when the fast-path connectivity checks fail. This occurs for inter-node communication and for intra-node peers that are not CUDA P2P-accessible. In this case, NVSHMEM still preserves the same offset-based addressing scheme described earlier, but the computed remote address can no longer be dereferenced directly by the local GPU. Instead, the operation is carried out through a network transport. Depending on system support, NVSHMEM either uses InfiniBand GPUDirect Async (IBGDA), if available, or a **host proxy** path, where a CPU thread executes the request on behalf of the GPU.

Network transport and connection management. Slow-path RMA relies on a pluggable remote-transport layer rather

than on a single network interface model. NVSHMEM can select among InfiniBand-oriented transports such as IBRC and IBDEVX, as well as higher-level backends such as UCX and libfabric. The InfiniBand-specific transports maintain PE-to-PE connectivity through Reliable Connection (RC) queue pairs (QPs), while IBGDA additionally supports Dynamic Connection (DC) queue pairs, whose endpoints can be switched on demand at the cost of extra steering overhead.

However, this RC-QP description does not apply universally. For example, on HPE Slingshot systems NVSHMEM uses the libfabric transport with the `cx_i` provider, where connectivity is established through libfabric endpoints and address vectors. UCX similarly constructs its own PE-to-PE endpoints from exchanged worker addresses. Recent NVSHMEM versions also expose QP-specific APIs to users, allowing finer control over queue-pair selection and usage when those InfiniBand-specific paths are active [10].

Device-side RMA On the device side, scalar put/get operations use helpers such as `nvshmemi_transfer_rma_p` and `nvshmemi_transfer_rma_g`, bulk operations use `nvshmemi_transfer_rma` and `nvshmemi_transfer_rma_nbi`, and related operations such as signal and atomics are handled through specialized helpers such as `nvshmemi_transfer_put_signal` and `nvshmemi_transfer_amo_*`. Each operation first checks whether IBGDA is available on the device. If so, the request is offloaded to device-initiated network routines such as `nvshmemi_ibgda_rma_*`, which construct and post RDMA work directly from GPU code to the NIC. Otherwise, the operation is encoded as a work request and written into a proxy buffer in host-pinned memory. A dedicated host proxy thread then consumes this descriptor and executes the corresponding network operation via functions such as `nvshmemi_proxy_rma_p`, `nvshmemi_proxy_rma_g`, and `nvshmemi_proxy_rma_nbi`. Thus, the slow path remains GPU-initiated at the API level, but completion is ultimately driven either by the NIC through IBGDA or by the host proxy through the descriptor queue.

Host-side RMA Host-side RMA APIs, such as `nvshmem_put`, delegate to the common helper `nvshmemi_prepare_and_post_rma`, which selects the appropriate transport. On the slow path, in short, the operation is treated as a remote network transfer.

In the off-stream case, NVSHMEM directly selects a remote transport with RMA capability, such as IBRC or libfabric, constructs the corresponding local and remote memory descriptors, and posts the operation through the transport’s host-side RMA entry point. Bulk transfers are handled through `nvshmemi_process_multisend_rma`, which posts the required sequence of RDMA operations. In the on-stream case, the request is instead encoded into the proxy command structure and launched on the specified CUDA stream through `nvshmemi_proxy_rma_launcher`, after which the host proxy thread drives the actual network operations..

The host-side slow path currently imposes several restrictions for remote transports. In particular, remote strided RMA

is not supported, host-side `g` operations are not implemented, and `put_signal` is supported only for on-stream execution via the proxy. These limitations reflect the fact that the current remote transport and proxy mechanisms are primarily optimized for contiguous transfers and device-side signaling.

VII. COLLECTIVE COMMUNICATION

Having established the mechanisms for symmetric memory and one-sided communication, we next examine NVSHMEM’s collective operations. These operations are built on top of the lower-level components discussed above.

NVSHMEM implements a variety of collective algorithms to target different message sizes and system configurations. Due to space constraints, we do not discuss each algorithm in detail. Instead, Table II summarizes the main designs and, based on our analysis, reports their communication volume and latency, measured by the number of synchronization steps. In the remainder of this section, we therefore focus on the core concepts that underlie NVSHMEM’s collective implementation rather than on each algorithm individually.

A. *pSync Buffer*

Each team in NVSHMEM owns a `pSync` buffer, short for **persistent synchronization buffer**, which is a symmetric memory region in device global memory that is used to coordinate collective operations, and in some cases point-to-point operations, across PEs.

To reduce interference, NVSHMEM lays out `pSync` regions in a strided manner across teams, so that synchronization state belonging to different teams does not fall on the same cache lines. Within each team’s `pSync` region, different collective operations such as Broadcast and Reduce are assigned fixed subregions using hardcoded offsets. This separation reduces cross-operation contention within a team and also simplifies device-side indexing. In addition, some operations are double-buffered so that consecutive invocations can alternate between buffers and avoid requiring an additional barrier between uses.

B. *Low Latency (LL) & LL128 Protocols*

To reduce the cost of explicit synchronization after data transfer, which can become a major bottleneck for small messages, NVSHMEM implements Low Latency (LL) and LL128 protocols, following a similar design to NCCL [5]. The goal of these protocols is to avoid expensive memory barriers on the critical path by coupling data movement with lightweight arrival notification.

In the LL protocol, each data unit is paired with a synchronization flag, and the sender packs two data elements together with two flags into a single 16-byte atomic write. On the receiver side, correctness is ensured by polling the flags: once they match the expected values, the receiver knows that the corresponding data has arrived and is ready to be consumed.

LL128 extends the same idea to improve bandwidth utilization. Instead of placing a flag after every 4 bytes as in regular LL, LL128 groups 120 bytes of data with an 8-byte

synchronization flag, forming a 128-byte transfer unit. This significantly reduces metadata overhead while preserving the same flag-based synchronization mechanism. However, LL128 can only be used safely over NVLink, since it relies on 128-byte atomic stores that are not generally guaranteed on other interconnects such as PCIe [5].

The LL and LL128 protocols require additional `pSync` storage, since the reception buffer must accommodate both payload data and the associated synchronization flags. As not all collective algorithms support these protocols, Table II highlights the ones that do.

C. *Support for Multiple Cooperative Thread Arrays (CTAs)*

One distinction is important at the outset. The explicit device-side collective APIs in NVSHMEM are threadgroup collectives: they provide thread-, warp-, and block-scoped entry points, but a single collective invocation is executed by one participating threadgroup rather than by multiple CTAs cooperating implicitly. This is consistent with NVSHMEM’s team model, where each team owns one set of internal collective resources such as `pSync` space.

The `Multi-CTA` column in Table II therefore refers only to the availability of a host-side `_on_stream` kernel family with an explicit multi-CTA launch path, typically implemented by duplicating team state through `team_dups` and partitioning the payload across CTAs. It does not describe the semantics of the explicit device-side collective API itself, which remains a single-threadgroup collective interface.

D. *Barrier and Synchronization*

NVSHMEM’s synchronization collectives are implemented separately from the data collectives in Table II. In the device code, both `nvshmem_sync` and `nvshmem_barrier` ultimately invoke a dissemination-style synchronization routine over the team’s `pSync` region. At each phase, a PE signals a set of partner PEs and then waits for the corresponding incoming notifications from another set of partners. The radix k is chosen from the runtime parameter `barrier_tg_dissem_kval` and clamped by team size. For block-scoped collectives on fully P2P-connected teams, the implementation may raise the radix to the team size.

Under this structure, synchronization requires $O(\log_k N)$ rounds for a team of N PEs, since arrival information propagates outward level by level. The difference between `sync` and `barrier` is that `barrier` additionally enforces communication completion and visibility before entering the dissemination rounds, via `quiet` or `__threadfence_system()` depending on the path, and may also enforce target-side consistency afterward. As a result, both primitives have the same synchronization depth, but `barrier` provides stronger ordering and completion guarantees.

VIII. BENCHMARKING

A. *Point-to-Point*

1) *Intra-node*: The most stable intra-node observation is that NVSHMEM’s scalar and bulk APIs occupy different parts

TABLE II

SUMMARY OF COLLECTIVE ALGORITHMS IN NVSHMEM (v3.3.9). M IS THE MESSAGE SIZE, N IS THE NUMBER OF PES, AND $S = \lceil M/B_{\text{seg}} \rceil$ DENOTES THE NUMBER OF SCRATCH-LIMITED SEGMENTS IN SEGMENTED ALLREDUCE. ALGORITHMS WHOSE NAMES CONTAIN NVLS RELY ON NVSWITCH AND NVLINK SHARP TO PERFORM IN-NETWORK OPERATIONS.

Collective	Algorithm	LL / LL128	Multi-CTA	Communication Volume	# Synchronizations
Broadcast	Brute-force put-to-all	No	✗	$O(MN)$	$O(1)$
	k -ary flat tree	LL only	✗	$O(MN)$	$O(\log_k N)$
	Topology-aware hierarchical tree	LL only	✗	$O(MN)$	$O(\log_k N_{\text{remote}} + \log_k N_{\text{intra}})$
AlltoAll	P2P / general all-push	No	✗	$O(MN^2)$	$O(1)$
FCollect (Allgather)	NVLS one-shot	LL only	✓	$O(MN^2)$	$O(1)$
	Generic all-push	Both	✓	$O(MN^2)$	$O(1)$
Reduce (AllReduce)	NVLS one-shot	No	✓	$O(MN^2)$	$O(1)$
	NVLS two-shot	No	✓	$O(MN)$	$O(1)$
	k -ary recursive exchange	No	✓	$O(MN \times k \log_k N)$	$O(\log_k N)$
	Hierarchical fcollect	Inherited from FCollect	✓	$O(MN^2)$	$O(1)$
	(Segmented) linear AllReduce	No	✓	$O(MN^2)$	Direct LD/ST: $O(1)$; Segmented: $O(SN)$
ReduceScatter	NVLS one-shot	No	✓	$O(MN^2)$	$O(1)$
	Generic all-push	No	✓	$O(MN^2)$	$O(1)$

of the design space. Scalar *g/p* operations provide lower latency for tiny transfers, while bulk *get/put* variants are the only practical way to approach high NVLink bandwidth. Bulk operations saturate with moderate thread and CTA counts; scalar operations require much more thread-level parallelism to approach the same throughput.

Equally important, threadgroup scope matters less once the transport becomes bandwidth-bound. For small transfers, startup and synchronization overhead dominate. For larger transfers, the relevant question becomes how quickly the kernel can generate enough in-flight work to saturate NVLink, not which individual API variant looks conceptually simpler.

The thesis measurements showed the same pattern across both TPB and CTA sweeps. Bulk operations approach saturation quickly, especially when each thread moves kilobytes rather than bytes. Scalar operations improve more gradually and remain more sensitive to thread count because their per-request overhead is less easily amortized. This is a useful rule of thumb for application writers: use scalar operations for fine-grained control, not as a substitute for bulk movement.

2) *Inter-node*: Inter-node behavior is qualitatively different. Bulk *get/put* can approach line-rate on the tested network, while scalar operations degrade by orders of magnitude when they fall onto proxy-dominated slow paths. In this regime, total message size matters more than per-thread payload, and adding more GPU threads often stops helping after only a few concurrent flows.

The implication is practical: NVSHMEM’s fine-grained API surface should not be interpreted as a license to issue fine-grained inter-node traffic indiscriminately. Inter-node use should favor aggregation, coalescing, and bulk movement wherever possible unless the platform provides a truly efficient GPU-initiated networking path.

The tested Slingshot-based environment is especially instructive because it highlights the difference between the programming model and the underlying transport. NVSHMEM still presents the same one-sided device API, but the measured

performance depends heavily on whether the platform supports native GPU-initiated progress or relies on a CPU proxy. This makes transport disclosure essential when comparing systems or reporting results.

B. Collective

1) *Methodology*: The underlying thesis evaluated NVSHMEM with its point-to-point and collective microbenchmarks across intra-node and inter-node settings, sweeping message size, threads per block, and CTA count. For collectives, the kernels repeatedly invoked the target operation inside a timed loop, with explicit warmup and barriers placed outside the measured region. Reported bandwidth distinguished algorithm bandwidth from bus bandwidth, which is important because all-to-all, allgather-like, and reduction-style collectives move different total traffic for the same user-visible payload.

For the paper, we focus on stable qualitative conclusions rather than reproducing every curve. The key advantage of this choice is that it lets us connect results back to implementation details: whether an operation is single-CTA, whether it relies on LL packing, and whether it executes mostly on direct P2P paths or transport-mediated paths.

2) *Intra-node*: The thesis data shows that NVSHMEM’s built-in intra-node collectives scale with message size as expected, but their peak throughput is often far below the NVLink roofline. The dominant reason is implementation structure rather than a flaw in the PGAS model itself: the default collective kernels are conservative and frequently rely on a single CTA, leaving substantial GPU and interconnect parallelism unused.

This is most visible in the all-to-all experiments. A custom multi-CTA implementation pushes bus bandwidth close to 150 GB/s, far above the default single-CTA behavior, and close to the expected NVLink/NVSwitch ceiling for the tested system. By contrast, allreduce remains much more limited, indicating that its current implementation is constrained by a combination of algorithmic structure, synchronization, and kernel configuration rather than raw link bandwidth alone.

The per-collective breakdown also matters. Broadcast and `fcollect` behave differently from allreduce because their communication patterns are easier to realize as straightforward pushes or multicast-like data motion. Allreduce, in contrast, pays more heavily for data dependencies, temporary storage, and repeated synchronization. This difference shows up even before comparing against NCCL.

3) *Inter-node*: Inter-node collectives inherit both network bottlenecks and the cost of the current collective implementation. As node count grows, bandwidth declines and then flattens once the fabric topology becomes the dominant limiter. This reinforces a broader lesson from the point-to-point results: inter-node NVSHMEM is highly sensitive to the exact path between the GPU and the network, and results must be interpreted through the transport actually in use.

Put differently, inter-node collective performance is a compound effect of three layers: the chosen collective algorithm, the amount of available GPU concurrency, and the actual network path. NVSHMEM exposes enough flexibility to help with the first two, but it cannot hide an unfavorable transport path.

4) *Comparison with NCCL*: The thesis also compared NVSHMEM against NCCL on systems where NCCL could exercise its mature collective kernels. The result was consistent across the tested all-to-all and allreduce settings: under similar launch conditions, NCCL reached higher peak throughput and sustained it over a wider range of message sizes. This is not surprising in hindsight. NCCL is built around multi-channel collective execution, topology-specific scheduling, and protocol engineering for exactly these workloads.

The more interesting result is where the gap comes from. It does not primarily reflect a weakness of one-sided communication. Rather, it reflects the fact that NVSHMEM’s built-in collectives are relatively thin realizations of collective semantics on top of a PGAS substrate, whereas NCCL is a purpose-built collective engine. Once the implementation is customized, for example with multi-CTA all-to-all kernels built directly on NVSHMEM primitives, a substantial fraction of the gap can be recovered for selected patterns.

This leads to a practical division of labor. If an application mostly needs standard dense collectives, NCCL remains the stronger default. If it needs GPU-initiated progress, neighborhood exchange, data-dependent communication, or communication fused deeply into a kernel, NVSHMEM becomes much more compelling even if its out-of-the-box collective throughput is lower.

IX. CASE STUDIES

A. Jacobi

We use NVIDIA’s multi-GPU Jacobi reference implementation as a compact scientific kernel study [25]. The main result is sobering but informative: NCCL consistently outperforms the NVSHMEM version for this workload once the problem size is large enough that communication is a meaningful fraction of the runtime.

The reason is not that NVSHMEM is categorically worse at halo exchange. Rather, this Jacobi code path exposes several of NVSHMEM’s current weak spots at once: frequent communication embedded inside a compute kernel, sensitivity to CTA shape, fragmented halo puts when parallelized naively, and limited payoff from increasing CTA concurrency without preserving overlap. The lesson is that NVSHMEM can simplify fused halo exchange code, but performance still depends on preserving efficient compute kernels and minimizing message fragmentation.

This case study is useful precisely because it is not synthetic. It shows that moving communication into the kernel is not automatically beneficial if the resulting kernel shape harms occupancy, cache locality, or synchronization structure. In other words, GPU-initiated communication is a powerful tool, but it still has to fit the surrounding compute kernel.

B. DeepEP

DeepEP illustrates a more successful pattern [26], [27]. It uses NVSHMEM primarily as a low-level internode substrate for expert-parallel dispatch and combine rather than as a high-level collective library. Initialization uses the unique-ID mode, RDMA buffers are allocated from the symmetric heap, and the actual kernels issue IBGDA-based one-sided operations directly from the device.

Two design choices are especially revealing. First, DeepEP builds a topology-aware two-level communication structure: internode communication is constrained to GPUs with the same local index, and NVLink handles the remaining intra-node fanout. Second, it separates bandwidth-oriented and low-latency paths, using more direct RDMA-centric kernels for latency-sensitive inference. In both cases, NVSHMEM functions as an enabler for custom communication kernels, which aligns with our overall conclusion that NVSHMEM is most compelling as a substrate for irregular and fused communication.

DeepEP also illustrates an important systems point: the most effective use of NVSHMEM may be below the application-facing API layer. Rather than exposing raw puts and gets to model code, a runtime can hide NVSHMEM inside topology-aware kernels and present a cleaner expert-parallel abstraction to the user. This is likely to be the dominant adoption pattern in future ML systems.

X. DISCUSSION

The paper’s central message is not that NVSHMEM beats or loses to NCCL in the abstract. The more useful framing is that the two systems optimize different levels of the stack. NCCL offers highly tuned bulk collectives. NVSHMEM offers a GPU-centric programming substrate for one-sided communication and custom protocols.

This distinction also explains NVSHMEM’s current limitations. Its weak ordering model gives performance but increases correctness burden. Its best performance depends heavily on NVIDIA-specific hardware features such as NVLink and GPU-initiated RDMA. Its built-in collectives are functional

and flexible, but not yet engineered to the same degree as NCCL’s production kernels. For future work, the most promising directions are better multi-CTA collective implementations, stronger support for GPU-initiated internode progress, and higher-level tooling that helps users reason about fences, quiet, and signal-based protocols.

Another open question is portability of programming style rather than API spelling. Other vendors now provide SHMEM-like interfaces, but the most effective NVSHMEM idioms often rely on NVIDIA-specific assumptions about threadgroup collectives, NVLink reachability, and device-initiated networking. A useful portability layer would therefore need to capture these execution patterns, not just rename API calls.

We also found that NVSHMEM is most persuasive when evaluated as a substrate and least persuasive when evaluated as a monolithic end-user library. The implementation and case studies both point in the same direction: higher-level runtimes can use NVSHMEM internally to realize communication structures that are awkward to express in MPI or NCCL, while shielding application authors from the weakest-ordering and transport-selection details. This suggests that future research should pay as much attention to software layering as to raw kernel throughput.

XI. CONCLUSION

NVSHMEM is best understood as a GPU-centric PGAS runtime rather than as “MPI for GPUs” or “NCCL with puts and gets.” Its most distinctive contribution is to move communication initiation and synchronization into device code, which makes it well suited for irregular communication, fine-grained overlap, and application-specific fused kernels. Our source-level analysis shows how this model is realized through symmetric memory, topology-aware transport setup, direct and transport-mediated communication paths, and a collection of collective kernels built atop the same substrate. Our benchmark and case-study synthesis shows both the payoff and the limits of the approach: NVSHMEM is a powerful systems building block, but its built-in collectives often leave performance on the table, and users must still design carefully around ordering, granularity, and topology. We hope this condensed study makes NVSHMEM easier to evaluate, use, and improve.

REFERENCES

- [1] T. Ben-Nun and T. Hoefer, “Demystifying parallel and distributed deep learning: An in-depth concurrency analysis,” *ACM Comput. Surv.*, vol. 52, Aug. 2019.
- [2] D. Valdez-Balderas, J. M. Domínguez, B. D. Rogers, and A. J. Crespo, “Towards accelerating smoothed particle hydrodynamics simulations for free-surface flows on multi-gpu clusters,” *Journal of Parallel and Distributed Computing*, vol. 73, no. 11, pp. 1483–1493, 2013. Novel architectures for high-performance computing.
- [3] J. Francés, B. Otero, S. Bleda, S. Gallego, C. Neipp, A. Márquez, and A. Beléndez, “Multi-gpu and multi-cpu accelerated ftd scheme for vibroacoustic applications,” *Computer Physics Communications*, vol. 191, pp. 43–51, 2015.
- [4] M. Punke and M. Salvalaglio, “Multi-gpu fast fourier transforms in matlab (for large-scale phase-field crystal simulations),” 2026.
- [5] Z. Hu, S. Shen, T. Bonato, S. Jeaugey, C. Alexander, E. Spada, J. Dinan, J. Hammond, and T. Hoefer, “Demystifying nccl: An in-depth analysis of gpu communication protocols and algorithms,” 2026.

- [6] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, “Pytorch: An imperative style, high-performance deep learning library,” 2019.
- [7] M. Shoenybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, “Megatron-lm: Training multi-billion parameter language models using model parallelism,” 2020.
- [8] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with pagedattention,” 2023.
- [9] K. Hamidouche, J. Bachan, P. Markthub, P.-J. Gootzen, E. Agostini, S. Jeaugey, A. Shafi, G. Theodorakis, and M. G. Venkata, “Gpu-initiated networking for nccl,” 2025.
- [10] NVIDIA, “NVSHMEM APIs,” 2025. Documentation verified against version 3.5.19.
- [11] N. Maruyama, B. V. Essen, J. Ciesko, J. Wilke, C. Trott, C.-H. Hsu, N. Imam, J. Dinan, A. Langer, C. Newburn, and S. Potluri, “Scaling scientific computing with nvshmem,” Aug. 2020. NVIDIA Developer Blog.
- [12] P. Markthub, J. Dinan, S. Potluri, and S. Howell, “Improving Network Performance of HPC Systems Using NVIDIA Magnum IO NVSHMEM and GPUDirect Async,” 2022. Section: InfiniBand GPUDirect Async.
- [13] S. Potluri, A. Goswami, D. Rossetti, C. J. Newburn, M. G. Venkata, and N. Imam, “GPU-Centric Communication on NVIDIA GPU Clusters with InfiniBand: A Case Study with OpenSHMEM,” in *2017 IEEE 24th International Conference on High Performance Computing (HiPC)*, pp. 253–262, 2017.
- [14] NVIDIA Corporation, “GPUDirect RDMA Documentation,” 2026. Accessed: 2026-04-22.
- [15] NVIDIA Corporation, “DOCA GPUNetIO Documentation (v3.2.2),” 2026. Last updated: March 23, 2026; Accessed: 2026-04-22.
- [16] G. Calin, E. Derevenetc, R. Majumdar, and R. Meyer, “A Theory of Partitioned Global Address Spaces,” *Leibniz International Proceedings in Informatics*, vol. 24, 2013.
- [17] B. Chapman, T. Curtis, S. Pophale, S. Poole, J. Kuehn, C. Koelbel, and L. Smith, “Introducing openshmem: Shmem for the pgas community,” in *Proceedings of the Fourth Conference on Partitioned Global Address Space Programming Model, PGAS ’10*, (New York, NY, USA), Association for Computing Machinery, 2010.
- [18] J. Kraus and J. Kraus, “An introduction to cuda-aware mpi,” *NVIDIA Technical Blog*, July 2025.
- [19] Message Passing Interface Forum, “MPI: A Message-Passing Interface Standard, Version 5.0,” 2025.
- [20] CLaSS Group at Lawrence Berkeley National Laboratory, “UPC++ Wiki.”
- [21] D. Bonachea and P. H. Hargrove, “GASNet-EX: A High-Performance, Portable Communication Library for Exascale,” in *Languages and Compilers for Parallel Computing (LCPC’18)*, Springer, 2018. Lawrence Berkeley National Laboratory Technical Report LBNL-2001174.
- [22] Advanced Micro Devices, Inc., “rocSHMEM - ROCm OpenSHMEM GPU library.”
- [23] A. Brooks, P. Marshall, D. Ozog, M. W. ur Rahman, L. Stewart, and R. Tom, “Intel(R) SHMEM: GPU-initiated OpenSHMEM using SYCL,” 2024.
- [24] C. Perry and N. Sakharnykh, “Introducing Low-Level GPU Virtual Memory Management,” 2020.
- [25] NVIDIA, “Multi GPU Jacobi solver with different multi GPU programming models,” 2025.
- [26] C. Zhao, S. Zhou, L. Zhang, C. Deng, Z. Xu, Y. Liu, K. Yu, J. Li, and L. Zhao, “Deepep: an efficient expert-parallel communication library,” 2025.
- [27] DeepSeek-AI, “Deepseek-v3 technical report,” 2025.

APPENDIX